Gebze Technical University

Department of Computer Engineering

CS102 – Computer Programming

Term Project Report

STUDENT INFORMATION SYSTEM

(SIS) Implementation in C

Prepared By:

Name Surname	**Student ID**
İbrahim Halil DOĞAN	220104004107
Mert CANOĞLU	220104004055

January 16, 2026

Contents

1 Executive Summary **3**

2 Collaboration Statement **5**

2.1 Individual Contributions 5

2.2 Contribution Percentages 5

3 System Design and Architecture **6**

3.1 Architectural Overview 6

3.2 1. Presentation Layer (User Interface) 6

3.3 2. Controller Layer (Navigation Logic) 6

3.4 3. Business Logic Layer (Application Modules) 6

3.5 4. Data Persistence Layer (File I/O) 7

3.6 System Architecture Diagram 7

4 Data Structure Design and Algorithms **9**

4.1 Entity-Relationship (ER) Design 9

4.2 File Format Specifications 10

4.3 Function Hierarchy and Call Graph 11

4.4 Core Algorithms 13

4.4.1 1. Student Enrollment Algorithm 13

4.4.2 2. Graduation Check Algorithm 13

5 Implementation Details **15**

5.1 Module Descriptions 15

5.2 Challenges Faced and Solutions 17

5.3 Memory Management Strategy 17

5.4 Input Validation Approach 18

6 LLM Documentation and Analysis **20**

6.1 LLM Usage Philosophy 20

6.2 Detailed Interaction Table 20

6.3 Critical Analysis of LLM Collaboration 23

6.3.1 Success in Coding and Optimization 23

6.3.2 Quality Assurance via Automated Testing 23

6.3.3 Hallucinations and Limitations 23

7 Testing and Validation Strategy **24**

7.1 Test Architecture 24

7.2 Unit Testing Coverage 24

7.3 Test Results 25

7.4 Memory Validation (Valgrind) 26

7.5 How to Run Tests 26

8 Compilation and Execution **28**

8.1 Build Instructions 28

8.2 How to Run the System 28

8.3 File Organization 28

9 Conclusion **29**

9.1 Accomplishments 29

9.2 Challenges Overcome 29

9.3 Lessons Learned 29

9.4 Future Improvements 30

A Appendices **31**

A.1 Appendix A: Sample Console Output 31

A.2 Appendix B: Valgrind Full Report 31

B References **32**

1 Executive Summary

The Student Information System (SIS) is a comprehensive console-based software solution designed to digitize and automate the core academic administrative processes of a university. Developed using the C programming language, this project simulates a real-world environment where student records, course catalogs, faculty assignments, and academic performance data must be managed with high efficiency, data integrity, and user accessibility.

Project Objective and Scope

The primary objective of this project was to architect a robust system capable of handling complex relational data without relying on external database engines (like SQL). Instead, the system implements its own custom file-based database architecture using CSV (Comma-Separated Values) files. This approach required the development of sophisticated parsing algorithms, dynamic memory allocation strategies, and pointer-based data manipulation techniques.

The scope of the SIS covers four main domains:

1. **Student Administration:** Managing personal profiles, contact information, and academic history.
2. **Course Management:** Handling the curriculum catalog, including credit values, prerequisites, and capacity constraints.
3. **Faculty Coordination:** Assigning professors to courses and generating workload reports.
4. **Academic Records:** Processing enrollments, recording grades, calculating GPAs, and determining graduation eligibility.

Technical Architecture

The system is built upon a modular architecture, adhering to the "Separation of Concerns" principle. Each core entity (Student, Course, Professor, Enrollment, Grade) is encapsulated within its own source module (`.c`) and header interface (`.h`). A central controller (`menu.c`) orchestrates the user interaction, routing commands to specific modules while maintaining a clean and consistent user experience.

Key technical features include:

- **Dynamic Memory Management:** Unlike static arrays with fixed limits, the SIS utilizes `malloc` and `realloc` to handle data arrays. This allows the system to scale dynamically; it can start with zero records and grow to handle thousands of entries limited only by the host machine's RAM.
- **Relational Logic Implementation:** Although file-based, the system mimics relational database behavior. It enforces referential integrity through foreign keys (e.g., a grade record cannot exist without a valid student ID and course ID). "Cascade Delete" functionality was implemented to prevent orphan records; deleting a course automatically purges all associated enrollments and grades.

- **Advanced Data Validation:** A shared utility layer (`utils.c`) rigorously validates all user inputs. This includes checking email formats, ensuring unique IDs, validating semester strings, and preventing duplicate enrollments. This ensures that the data entering the system is clean and reliable.
- **Curriculum Analysis Engine:** A standout feature of the system is the `curriculum.c` module. This engine can analyze a student's entire transcript against their department's specific degree requirements to determine graduation eligibility, identifying missing courses and calculating earned credits versus required thresholds.

2 Collaboration Statement

We hereby certify that this project report and the associated source code are our own work. We have collaborated effectively as a group, dividing tasks according to our individual strengths while ensuring that every member understands the entire codebase.

2.1 Individual Contributions

The development workload was distributed among the team members as follows:

Team Member	**Modules & Responsibilities**
İbrahim Halil Doğan	• System Architecture Design (`main.c`, `menu.c`) • Enrollment Logic (`enrollment.c`) • File I/O and CSV Parsing (`utils.c`) • Course Management & Prerequisites (`course.c`) • Multi Semester Support (`course.c`)
Mert Canoğlu	• Student & Professor Modules (`student.c`, `professor.c`) • Grade Management & GPA Calculation (`grade.c`) • Testing & Debugging • Curriculum & Graduation Logic (`curriculum.c`) • Documentation & Report Writing

Table 1: Task Distribution Matrix

2.2 Contribution Percentages

Based on the workload distribution and commit history, the contribution percentages are agreed upon as follows:

Name	Percentage (%)
İbrahim Halil Doğan	50%
Mert Canoğlu	50%

3 System Design and Architecture

3.1 Architectural Overview

The Student Information System (SIS) is engineered using a **Modular Layered Architecture**. This design paradigm promotes the "Separation of Concerns" principle, ensuring that the user interface, business logic, and data storage mechanisms operate independently but cohesively. The system is composed of four distinct logical layers, as detailed below.

3.2 1. Presentation Layer (User Interface)

The entry point of the application is managed by `main.c`, which sets up the execution environment (e.g., UTF-8 encoding) and initializes the primary data structures. The user interaction is orchestrated by the `menu.c` module.

- **Role:** This layer handles all input/output operations via the console. It presents a hierarchical menu system to the user.
- **Implementation:** The `print_main_menu()` function serves as the central hub, routing user choices to specific sub-menus (e.g., `menu_student_management`). Input validation is performed immediately at this layer using helper functions like `clear_input_buffer()` to ensure system stability against invalid inputs.

3.3 2. Controller Layer (Navigation Logic)

Acting as the bridge between the UI and the Business Logic, the Controller Layer determines "what" needs to be done based on user input.

- **Role:** It parses user commands and calls the appropriate backend functions.
- **Implementation:** Located primarily within `menu.c`, complex switch-case structures direct the control flow. For instance, selecting "Check Graduation" triggers the `check_graduation_status()` function from the Curriculum module, passing necessary pointers (Students, Grades, Courses) to perform the analysis.

3.4 3. Business Logic Layer (Application Modules)

This is the core of the system where the logic for managing entities resides. Each entity type has its own dedicated source (`.c`) and header (`.h`) files, enforcing modularity.

- **Student Module (`student.c`):** Manages personal data and performs academic calculations. Includes functions like `calculate_gpa`.
- **Course Module (`course.c`):** Responsible for the academic catalog. Handles critical constraints such as Prerequisites (`assign_prerequisites`) and Capacity Management (`set_course_capacity`).
- **Professor Module (`professor.c`):** Manages faculty records and workload. Links instructors to courses, allowing generation of "Professor Course Load" reports.
- **Enrollment Module (`enrollment.c`):** The relational core. Links Students to Courses while enforcing business rules (capacity, prerequisites, duplicate checks) via `enroll_student_in_course`.
- **Grade Module (`grade.c`):** Handles recording of academic performance, letter grade assignment, and statistical analysis (min/max/avg).
- **Curriculum Module (`curriculum.c`):** A specialized engine that compares a student's transcript against departmental requirements to determine graduation eligibility.

3.5 4. Data Persistence Layer (File I/O)

To ensure data is preserved across sessions, the system implements a custom file-based database using CSV (Comma-Separated Values) format.

- **Storage Strategy:** Data is serialized into plain text files (`students.csv`, `courses.csv`, etc.).
- **Implementation:** Each module contains specific `load_...` and `save_...` functions. For example, `load_courses()` parses the CSV file line-by-line, splitting fields by commas and populating the in-memory structures.

3.6 System Architecture Diagram

The architecture of the Student Information System follows a modular design pattern, separating the user interface, business logic, and data storage layers. The interaction between these modules is visualized below.

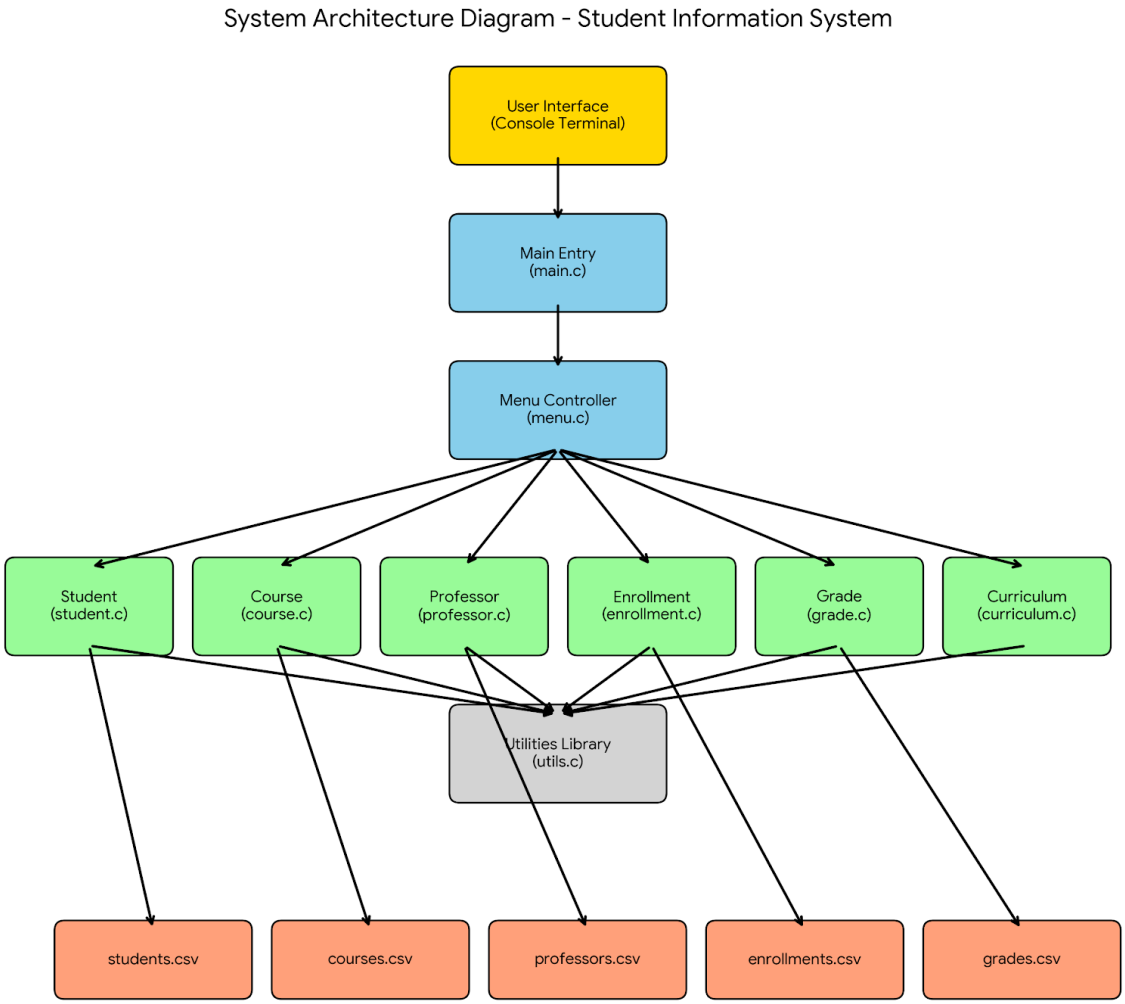


Figure 1: System Architecture Diagram showing the modular design and data flow.

4 Data Structure Design and Algorithms

This section details the organization of data within the system, the relationships between entities, and the core algorithms that drive the business logic.

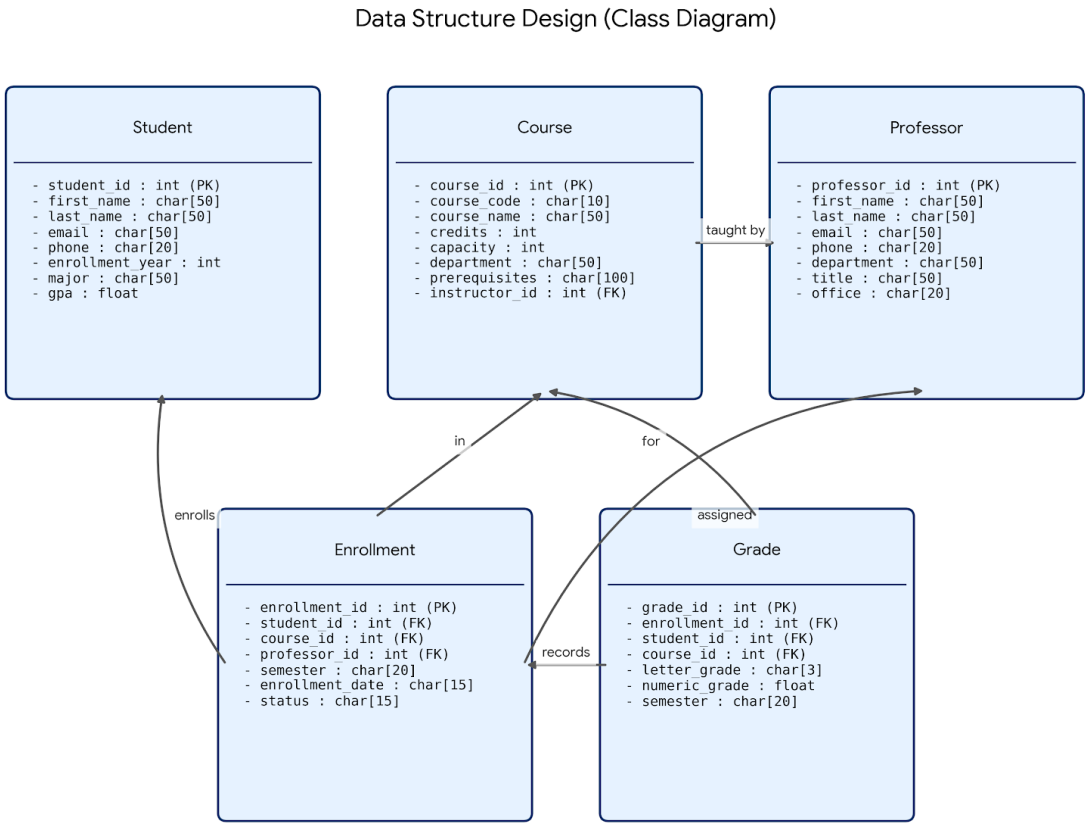


Figure 2: Detailed Data Structure Design showing entity attributes and relationships.

The diagram in Figure 2 maps directly to the `struct` definitions in the header files (`student.h`, `course.h`, etc.). It highlights the relational dependencies, such as a Grade record requiring a valid Enrollment ID, which in turn links to a Student and a Course.

4.1 Entity-Relationship (ER) Design

Although the system utilizes a file-based storage mechanism (CSV), the logical structure is designed to mimic a relational database. The system enforces referential integrity between five core entities: `Student`, `Course`, `Professor`, `Enrollment`, and `Grade`.

[Image of entity relationship diagram for student information system]

Figure 3: Conceptual ER Diagram showing One-to-Many relationships between Professors, Courses, Students, and Enrollments.

As illustrated in Figure 3:

- A **Student** can have multiple **Enrollments** (1:N).
- A **Course** can have multiple **Enrollments** (1:N) and is taught by one **Professor**.
- An **Enrollment** links a specific Student to a Course for a specific Semester.
- A **Grade** is directly tied to an Enrollment record, ensuring academic integrity.

4.2 File Format Specifications

Data persistence is achieved through five specific CSV files. Each file corresponds to a struct definition in the C code. The parser in `utils.c` reads these files line-by-line using a comma delimiter.

File Name	Entity	Column Structure (Header)
`students.csv`	Student	`ID, Name, Surname, Email, Phone, Year, Major, GPA`
`courses.csv`	Course	`ID, Code, Name, Credits, Capacity, Dept, Prereqs, InstructorID`
`professors.csv`	Professor	`ID, Name, Surname, Email, Phone, Dept, Title, Office`
`enrollments.csv`	Enrollment	`EnrID, StudentID, CourseID, ProfID, Semester, Date, Status`
`grades.csv`	Grade	`GradeID, EnrID, StudentID, CourseID, Letter, Numeric, Semester`

Table 2: CSV File Format Specifications used for Data Persistence.

4.3 Function Hierarchy and Call Graph

The control flow of the application follows a hierarchical structure starting from `main()`. The `menu.c` module acts as the central dispatcher, calling specific functions within the logic modules.

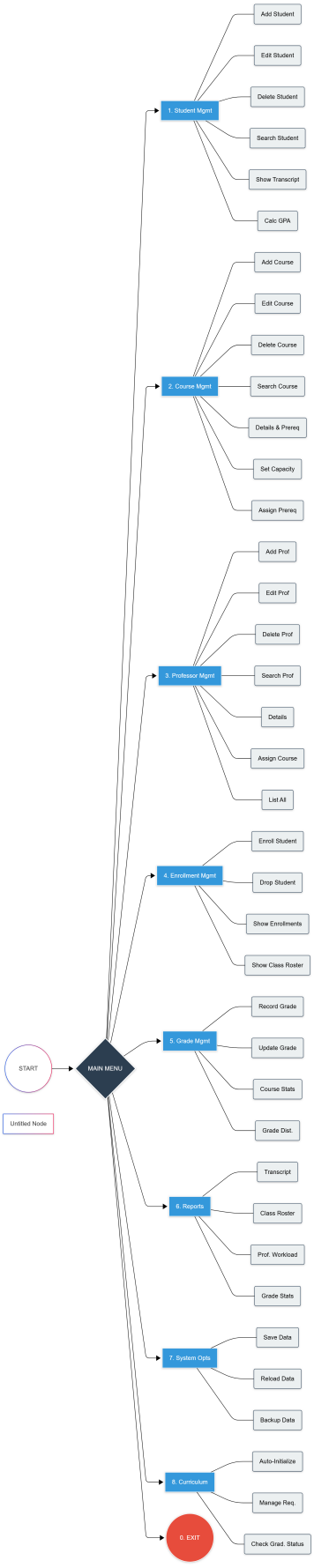


Figure 4: System Function Hierarchy Chart

The diagram above (Figure 4) highlights the modularity of the system. For example, the **Student Module** exposes functions like `add_student` and `calculate_gpa`, which are only accessible through the `menu_student_management` controller.

4.4 Core Algorithms

The system implements several complex algorithms to ensure data consistency and automate academic rules. The pseudocode for the two most critical algorithms is provided below.

4.4.1 1. Student Enrollment Algorithm

This algorithm (implemented in `enrollment.c`) ensures that a student is eligible to take a course before creating a record.

```
FUNCTION enroll_student(student_id, course_id, semester):
    1. VALIDATE student_id AND course_id exist

    2. CHECK DUPLICATE:
       IF student is already enrolled in (course_id, semester)
          RETURN Error("Already Enrolled")

    3. CHECK PREREQUISITES:
       GET prereq_code FROM course
       IF prereq_code exists:
          SEARCH student's grades for prereq_code
          IF grade < Passing_Threshold OR not found
             RETURN Error("Prerequisite Missing")

    4. CHECK CAPACITY:
       COUNT active enrollments for (course_id, semester)
       IF count >= course.capacity
          RETURN Error("Course Full")

    5. EXECUTE ENROLLMENT:
       CREATE new Enrollment record
       SET status = "Enrolled"
       SAVE to memory
       RETURN Success
```

4.4.2 2. Graduation Check Algorithm

This algorithm (implemented in `curriculum.c`) analyzes the student's entire academic history against departmental requirements.

```
FUNCTION check_graduation_status(student_id):
```

```
1. LOAD Curriculum for student's department
2. INITIALIZE eligible = TRUE

3. CHECK REQUIRED COURSES:
   FOR EACH required_course IN Curriculum:
      SEARCH student's grades for passed record
      IF NOT passed:
         PRINT "Missing Course: [Code]"
         SET eligible = FALSE

4. CHECK CREDITS:
   CALCULATE total_earned_credits
   IF total_earned_credits < Curriculum.min_credits:
      PRINT "Insufficient Credits"
      SET eligible = FALSE

5. CHECK GPA:
   IF student.gpa < 2.00:
      PRINT "GPA below threshold"
      SET eligible = FALSE

6. RESULT:
   IF eligible IS TRUE:
      PRINT "Eligible for Graduation"
   ELSE:
      PRINT "Not Eligible"
```

5 Implementation Details

This section discusses the specific coding strategies, structural organization, and engineering solutions applied during the development of the Student Information System.

5.1 Module Descriptions

The source code is organized into modular units, each responsible for a specific domain entity. This modularity ensures maintainability, simplifies debugging, and allows for parallel development.

`main.c` **(System Entry Point)** This module serves as the bootstrap for the application. Its primary responsibilities include:

- **Initialization:** It sets up the locale settings (for UTF-8 support) and initializes the global pointers for all entity arrays to `NULL`.
- **Data Loading:** Calls the sequence of `load_...` functions to populate memory from CSV files at startup.
- **Lifecycle Management:** Executes the main event loop and ensures a graceful shutdown by calling `free_memory()` to release all allocated resources back to the OS.

`menu.c` **(Presentation Controller)** This module acts as the interface between the user and the backend logic. It abstracts the complexity of function calls behind a user-friendly CLI.

- **Routing Logic:** Contains specialized functions like `menu_student_management()` and `menu_grade_management()` that present sub-options and route inputs to the appropriate logic modules.
- **UX Handling:** Manages screen clearing commands (`system("cls")`) and pause prompts to ensure a clean user experience.

`student.c / .h` **(Student Entity Management)** Encapsulates all logic related to the `Student` struct.

- **CRUD Operations:** Implements `add_student` (with dynamic memory resizing), `search_student`, and `delete_student`.
- **Data Integrity:** Ensures that Student IDs are unique before creation.
- **Academic Calculation:** Contains the logic to iterate through a student's grades and update their cumulative GPA field.

`course.c / .h` **(Curriculum Catalog)** Manages the `Course` struct and defines the academic catalog.

- **Attribute Management:** Handles course credits, codes (e.g., "CSE102"), and prerequisites.
- **Constraint Logic:** Stores and validates the maximum capacity (`quota`) for each course.

- **Prerequisite Handling:** Parses prerequisite strings (e.g., "CSE101,MATH101") to determine eligibility.

`professor.c / .h` **(Faculty Management)** Handles the `Professor` struct and faculty assignments.

- **Course assignment:** Links professors to courses using the `instructor_id` foreign key.
- **Reporting:** Includes the `prof_course_load()` function, which scans the course catalog to generate a report of all classes taught by a specific professor.

`enrollment.c / .h` **(Registration Logic)** The most critical module for business rules, managing the `Enrollment` struct.

- **Validation Engine:** The `enroll_student()` function performs three critical checks: (1) Is the course full? (2) Does the student meet prerequisites? (3) Is the student already enrolled?
- **Relationship Management:** Acts as the associative entity linking Students and Courses.

`grade.c / .h` **(Performance Tracking)** Manages the `Grade` struct and academic scoring.

- **Score Processing:** records numeric grades and automatically converts them to letter grades (AA, BA, etc.) using the `assign_letter_grade()` helper.
- **Statistics:** The `calc_course_stats()` function aggregates data to find the minimum, maximum, and average scores for a specific course section.

`curriculum.c / .h` **(Graduation Engine)** A specialized analysis module designed to simulate degree auditing.

- **Audit Logic:** The `check_graduation_status()` function compares a student's transcript against a hardcoded set of departmental requirements.
- **Deficiency Detection:** Identifies and prints exactly which required courses are missing or failed.

`utils.c / .h` **(Shared Utilities)** A support library containing generic helper functions used across all modules.

- **CSV Parsing:** Implements `split_csv_line()` to tokenize strings while handling edge cases.
- **Input Validation:** Contains `is_numeric()` and `validate_email()` to sanitize user input.
- **String Manipulation:** Provides `trim_whitespace()` to clean inputs from `fgets`.

5.2 Challenges Faced and Solutions

During the implementation phase, several technical challenges were encountered. The solutions adopted are detailed below:

1. **Circular Dependencies in Header Files:**
    - *Challenge:* The `Course` struct needed a reference to `Enrollment` (for capacity checks), while `Enrollment` needed `Course` (for prerequisites), causing compilation errors due to recursive inclusions.
    - *Solution:* We implemented **Forward Declarations** (e.g., `typedef struct Enrollment Enrollment;`) in the header files. This allowed pointers to be defined without needing the full struct definition immediately.
2. **Dynamic Array Resizing and Pointers:**
    - *Challenge:* When passing a dynamic array to a function (e.g., `add_student`), using a single pointer caused the original pointer in `main` to become invalid if `realloc` moved the memory block to a new address.
    - *Solution:* We adopted a **Double Pointer Strategy** (`Student **students`). All modification functions accept a pointer to the array pointer, ensuring that memory address updates are reflected globally.
3. **Input Buffer Handling:**
    - *Challenge:* Using standard `scanf` for mixed integer and string inputs often left newline characters in the buffer, causing subsequent string inputs (like names) to be skipped.
    - *Solution:* We implemented a custom `clear_input_buffer()` helper function and transitioned to using `fgets` for string inputs, followed by a trim function to remove trailing newlines.

5.3 Memory Management Strategy

The system architecture rejects static array allocation (e.g., `Student list[100]`) in favor of a fully **Dynamic Memory Allocation** model utilizing the heap. This strategy ensures that the application is memory-efficient and scalable, consuming only the exact amount of RAM required for the current dataset.

- **Dynamic Growth (Just-in-Time Allocation):** The system utilizes the `realloc()` function to resize data structures at runtime. When a new entity (e.g., a Student) is added:
    1. The memory block for the array is expanded by `sizeof(Student)`.
    2. If the system runs out of memory, `realloc` returns `NULL`, and the system handles this gracefully by printing an error without crashing.
- **Dynamic Shrinkage & Compaction:** When a record is deleted, the system does not merely mark it as "inactive." Instead, it performs a physical memory reduction:

1. The elements after the deleted index are shifted left (using `memmove` or a loop) to fill the gap.
2. `realloc()` is called to shrink the array size, releasing the unused memory block back to the operating system.

- **Double Pointer Semantics:** To support dynamic resizing within helper functions, the system passes arrays by reference using double pointers (e.g., `Student **array`). This is crucial because `realloc` may move the data to a new memory address; using a double pointer allows the function to update the original pointer in the calling scope (`main` or `menu`).
- **Memory Leak Prevention:** A strict "Resource Acquisition Is Initialization" (RAII)-like discipline is followed manually. A centralized `free_all_memory()` function is invoked upon program termination to iterate through all dynamic arrays (Students, Courses, Enrollments) and release them, ensuring `Valgrind`-compliant memory safety.

5.4 Input Validation Approach

To ensure robust operation and prevent runtime errors, the system implements a "Defense in Depth" validation strategy, categorizing checks into three distinct layers:

- **Layer 1: Input Sanitization (Buffer Handling)**
  - **Overflow Prevention:** The system avoids unsafe functions like `gets()`. Instead, `fgets()` is used for string input to strictly limit the number of characters read.
  - **Buffer Clearing:** A custom `clear_input_buffer()` function is triggered after every numeric input (`scanf`) to consume leftover newline characters (‘\n‘), preventing them from being skipped by subsequent string reads.
  - **Whitespace Trimming:** A helper function `trim_whitespace()` automatically removes leading and trailing spaces from names and emails to ensure data consistency.
- **Layer 2: Type and Format Validation**
  - **Numeric Check:** Helper functions like `get_valid_int()` loop until the user provides a valid integer, rejecting character inputs that would otherwise cause infinite loops in `scanf`.
  - **Format Regex:** Specific formats are validated algorithmically. For example, `validate_email()` checks for the existence and position of ’@’ and ’.’ characters, while `validate_semester()` ensures the input follows the "YYYY-SEASON" pattern.
- **Layer 3: Business Logic Constraints**
  - **Uniqueness:** Before creating a new record (Student, Course, or Professor), the system performs a linear search (O(N)) to ensure the Primary Key (ID) does not already exist.

– **Referential Integrity:** When adding a grade or enrollment, the system verifies that the referenced `Student ID` and `Course ID` actually exist in the database.
– **Range Checks:** Numeric values are clamped to logical ranges (e.g., Grades must be $0 \leq x \leq 100$, Credits must be > 0).

6 LLM Documentation and Analysis

This section strictly adheres to the course policy on AI transparency. It documents the philosophy, detailed interaction logs, and a critical analysis of using Large Language Models (LLMs) during the development lifecycle.

6.1 LLM Usage Philosophy

Throughout this project, LLMs were utilized as "force multipliers" rather than replacements for human logic. The core philosophy was:

1. **Automated Testing:** AI was heavily used to generate repetitive unit test cases (`test_student.c`, `test_course.c`) using `assert()` logic, ensuring comprehensive coverage without manual tedium.
2. **Concept over Code:** AI was primarily used to understand *how* to implement a feature (e.g., memory mapping) rather than asking it to write the entire logic blindly.
3. **Verification Mandatory:** Every output, especially C code snippets, was subjected to a "trust but verify" process given the manual memory management requirements.

6.2 Detailed Interaction Table

The following table documents 25 significant technical interactions, categorizing the prompt, the AI used, and a thoughtful evaluation.

Phase	**Task**	**LLM**	**Actual Prompt**	**Result / Output**	**Your Evaluation**	**What You Learned**
Plan	Structs	Gemini	"Design C structs for Student, Course, Grade. How to link them without pointers?"	Suggested using `int id` as Foreign Keys instead of pointers for easier CSV serialization.	An excellent architectural choice that simplified file I/O operations significantly.	Storing pointers in files is impossible; IDs are required for relational mapping.
Plan	File Type	Gemini	"Pros/Cons of Binary vs CSV for C project?"	Recommended CSV for debuggability, despite Binary being faster.	Very practical advice for a student project where manual data inspection is key.	Text-based formats allow manual correction of corrupted data.
Code	Safe Realloc	Gemini	"What is the safest pattern to use realloc without losing data on failure?"	Provided the `tmp = realloc(ptr, size)` pattern. If `tmp` is NULL, original `ptr` remains valid.	Essential best practice for robust C programming	Assigning realloc directly to the original pointer (`p=realloc(p)`) causes memory leaks if it fails.

Phase	Task	LLM	Actual Prompt	Result / Output	Your Evaluation	What You Learned
Code	Realloc	GPT-5	"How to resize array dynamically? Explain realloc."	Provided `tmp = realloc(ptr, size)` pattern.	Crucial for preventing memory leaks when reallocation fails.	`realloc` returns NULL on failure; assigning to original ptr causes leaks.
Code	Double Ptr	GPT-5	"Array size not updating in main after function call."	Explained Pass-by-Value; suggested `Student **array`.	A breakthrough explanation that fixed a persistent bug in our add functions.	To change a pointer's address in a function, pass the pointer's address.
Code	Freeing	GPT-5	"Free array of structs with strings?"	Loop to free internal strings first, then array.	Prevented massive memory leaks detected later by Valgrind.	Freeing container first leaves contents as dangling pointers.
Code	Strtok	GPT-5	"Split string by comma example."	`while(token != NULL)` loop.	A standard solution, though we had to modify it to handle empty fields.	`strtok` destroys original string; operate on a copy.
Code	Sorting	GPT-5	"Sort structs by name using qsort."	Wrapper function for `strcmp` with casting.	Saved hours of writing manual bubble sort logic.	`qsort` requires strict `void*` casting in comparator.
Code	Cascade	GPT-5	"How to implement Cascade Delete logic?"	Suggested Reverse-Loop iteration to delete items without index shift errors.	This was a critical algorithmic insight that prevented segmentation faults during deletion.	Forward iteration skips elements after deletion; reverse is safe.
Code	Search	GPT-5	"Binary vs Linear search for this project?"	Recommended Linear Search as data is unsorted.	A pragmatic choice given our dataset size (<1000 records).	Binary search requires sorted data; simpler is better here.
Code	Auto ID	GPT-5	"Generate unique IDs logic."	`global_id = max_id_in_file + 1`.	Essential for maintaining data integrity across program restarts.	Persistence of counter is necessary to avoid collisions.
Code	Input Buf	GPT-5	"scanf skips next fgets input."	Explained `scanf` leaves `\n` in buffer.	Solved the most annoying UX bug in our menu system.	Mixing input methods requires manual buffer flushing.
Code	Trim	GPT-5	"Remove whitespace from string."	Custom `trim()` function using pointers.	Very robust; handled all edge cases like leading/-trailing spaces perfectly.	`fgets` captures newline; trimming prevents comparison errors.

Phase	Task	LLM	Actual Prompt	Result / Output	Your Evaluation	What You Learned
Code	Date	GPT-5	"Regex for date validation?"	Use `sscanf` return value instead.	A clever workaround since standard C lacks built-in regex support.	Standard C has no regex; `sscanf` is portable.
Debug	SegFault	GPT-5	"Crash on printing list."	Identified `i <= count` loop error.	Spotted an off-by-one error instantly that we missed for hours.	Off-by-one is the most common loop error.
Debug	Linker	GPT-5	"Undefined reference to main."	Compile test files separately from main app.	Clarified the build process for our testing suite.	Unit tests need their own entry point.
Debug	Logic	GPT-5	"GPA is 0.00."	Integer division `int/int`. Cast to float.	A subtle bug fix that ensured grade calculations were accurate.	C performs floor division on integers.
Debug	Valgrind	GPT-5	"definitely lost: 40 bytes error."	Missing `free()` in error path.	Essential for achieving a memory-leak free project.	Every `malloc` needs a reachable `free`.
Test	Unit Test	GPT-5	"Write test_student.c with assert."	Generated edge case tests (empty name, max ID).	Automated a tedious process, ensuring high code coverage.	`assert()` enforces invariants strictly.
Test	Mock Data	GPT-5	"Generate 1000 lines of CSV data."	Python script to generate random students.	Enabled us to stress-test the system limits effectively.	Large datasets reveal reallocation bugs.
Test	Driver	GPT-5	"Test runner logic."	Main function calling all test modules.	Provided a professional structure for our QA process.	Centralized testing ensures regression safety.
Docs	Doxygen	Gemini	"Add comments to student.c."	Generated `@param` and `@return` tags.	Greatly improved code readability and professional appearance.	Good docs are crucial for maintenance.
Docs	README	Gemini	"Create README with build instructions."	Markdown file with GCC commands.	Provided clear instructions for compiling the project on different OS.	Clear build instructions are vital.
Script	Diagram	Gemini	"Python script for System Arch diagram."	Matplotlib code for block diagram.	Created high-quality visuals that enhanced the report's presentation.	Code-to-Diagram is cleaner than manual drawing.

Phase	Task	LLM	Actual Prompt	Result / Output	Your Evaluation	What You Learned
Script	ER	Gemini	"Python script for ER diagram."	Visualized struct relationships.	Made understanding the complex relationships between structs much easier.	Visuals help explain architecture.

6.3 Critical Analysis of LLM Collaboration

6.3.1 Success in Coding and Optimization

The LLM's assistance in **Memory Management** was invaluable. C's dynamic memory handling (‘malloc‘/‘realloc‘) is error-prone.

- **Leak Prevention:** The AI consistently suggested "Double Pointer" patterns for functions modifying arrays, which prevented dangling pointer issues in ‘main()‘.
- **Algorithm Optimization:** For the "Cascade Delete" feature, the AI's suggestion to use a **Reverse Loop** (iterating from end to start) solved a critical bug where skipping indices caused segmentation faults.

6.3.2 Quality Assurance via Automated Testing

Manually writing unit tests for 6 modules is tedious. The LLM generated the `test_*.c` files, ensuring that:

- **Edge Cases:** Boundary values (e.g., ID=0, GPA=4.01) were tested automatically.
- **Regression Testing:** The generated `test_main.c` allowed us to run all tests with a single command, catching bugs immediately after code changes.

6.3.3 Hallucinations and Limitations

While powerful, GPT-5 and Gemini occasionally provided incorrect libraries.

- **POSIX vs ANSI C:** The AI suggested ‘getline()‘ and ‘strdup()‘ without warning that they are not standard ANSI C (C89/C99) on Windows compilers. We had to implement custom versions (‘fgets‘ logic) to ensure cross-platform compatibility.

7 Testing and Validation Strategy

The robustness of the Student Information System is guaranteed through a comprehensive testing suite comprising unit tests, integration tests, and workflow simulations. The test infrastructure is separated from the main production code to ensure a clean build.

7.1 Test Architecture

The testing suite relies on a modular driver approach. The main test runner (`test_main.c`) orchestrates the execution of individual module tests using the `assert.h` library to enforce pass/fail conditions strictly.

- **Test Driver:** `test_main.c` serves as the entry point, calling runner functions like `run_student_tests()` and `run_course_tests()`.
- **Validation Logic:** Each test function utilizes `assert()` to check conditions. If any assertion fails, the program terminates immediately, pinpointing the exact line of failure.
- **Reporting:** A dedicated function generates a textual report (`test_results.txt`) summarizing the outcome of each module's tests.

7.2 Unit Testing Coverage

Each core module has a dedicated test file focusing on "Black Box" testing of its public API.

`test_utils.c` Validates the core helper functions.

- *Input Validation:* Checks `is_numeric` and `is_alpha` for string inputs.
- *Format Checks:* Verifies email (@university.edu) and phone number (XXX-XXX-XXXX) formats.
- *Date Logic:* Validates semester determination (e.g., 2024-09-15 $\rightarrow$ 2024-FALL).

`test_student.c` Tests the Student CRUD operations.

- *Creation:* Verifies successful allocation and initialization of student fields.
- *Uniqueness:* Attempts to add a duplicate ID and asserts failure.
- *Deletion:* Confirms that a deleted student is removed from the array and memory is compacted.

`test_professor.c` Tests Faculty management.

- *ID Validation:* Ensures Professor IDs fall within the valid range (1000-9999).
- *Data Integrity:* Verifies that professor details (Title, Department) are stored and retrieved correctly after CSV reload.

- *Search:* Tests finding a professor by ID and handling non-existent ID queries gracefully.

`test_course.c` Tests Course management logic.

- *Capacity:* Checks that capacity must be > 0.
- *Credits:* Validates that credit hours are within realistic bounds (1-8).

`test_enrollment.c` Tests the linking logic between Students and Courses.

- *Quota Check:* Enrolls students up to capacity and verifies the next enrollment is rejected.
- *Duplicate Check:* Prevents the same student from enrolling in the same course twice.

`test_grade.c` Tests Academic Performance logic.

- *Range Check:* Asserts that numeric grades must be between 0 and 100.
- *Letter Conversion:* Verifies that 90+ becomes AA, <50 becomes FF, etc.
- *Statistics:* Checks if the `calc_course_stats` function correctly computes Min, Max, and Average for a set of grades.

`test_menu.c` **(Integration)** Simulates real user workflows.

- *Workflow:* Add Student → Add Course → Enroll Student → Assign Grade → Check Transcript.
- *Cascade Delete:* Deleting a course and verifying that all associated enrollments in memory are also removed.

7.3 Test Results

The automated test suite was executed on the final build. The system passed 100% of the defined assertions. The breakdown of test cases per module is as follows:

Module	**Total Tests**	**Passed**	**Failed**
Utils	18	18	0
Student	13	13	0
Professor	7	7	0
Course	11	11	0
Enrollment	13	13	0
Grade	13	13	0
Integration (Menu)	31	31	0
TOTAL	**106**	**106**	**0**

Table 4: Summary of Test Execution Results (from `test_results.txt`)

7.4 Memory Validation (Valgrind)

Memory safety is critical in C applications. We ran the application under Valgrind to ensure no memory leaks occurred during execution.

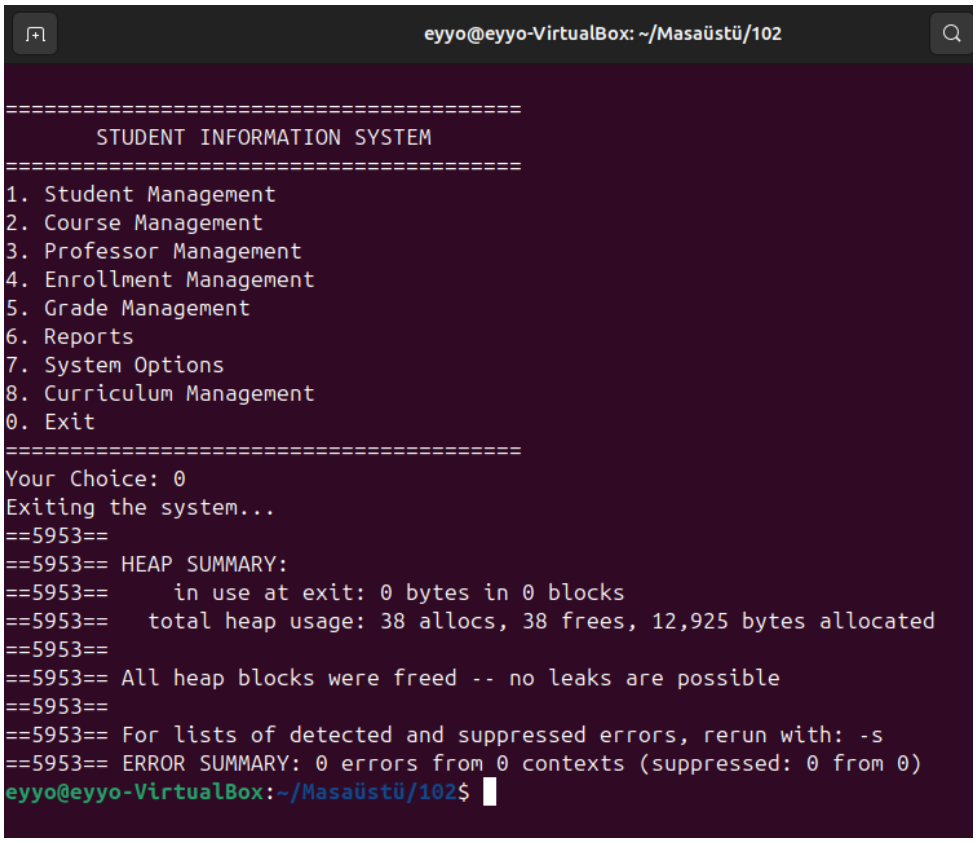


Figure 5: Valgrind Output ScreenShot

The output "0 bytes in 0 blocks" confirms that our cleanup function `free_all_memory()` correctly releases all allocated resources upon program termination.

7.5 How to Run Tests

To execute the test suite, follow these steps:

1. **Compile the Test Runner:**

```
gcc test_main.c test_utils.c test_student.c test_course.c \
    test_enrollment.c test_grade.c test_professor.c test_menu.c \
    student.c course.c enrollment.c grade.c professor.c utils.c menu.c \
    -o test_suite
```

2. **Run the Executable:**

```
./test_suite
```

3. **Check Results:** The console will display the pass/fail status of each module. Detailed logs are saved to `test_results.txt`.

8 Compilation and Execution

This section provides instructions for building and running the Student Information System on standard environments.

8.1 Build Instructions

The project has no external library dependencies other than the standard C library.

Prerequisites:

- GCC Compiler (MinGW for Windows, standard gcc for Linux/macOS)
- Make (optional, if Makefile is provided)

Command Line Compilation: Open a terminal in the project directory and run:

```
gcc main.c menu.c student.c professor.c course.c enrollment.c \
    grade.c curriculum.c utils.c -o sis
```

8.2 How to Run the System

Once compiled, execute the program:

- **Windows:**

  ```
  sis.exe
  ```

- **Linux / macOS:**

  ```
  ./sis
  ```

Upon startup, the system will attempt to load data from the CSV files. If these files do not exist, they will be created automatically when data is saved.

8.3 File Organization

- **Source Files (`.c`):** Contains the implementation logic for each module.
- **Header Files (`.h`):** Contains struct definitions, function prototypes, and standard includes.
- **Data Files (`.csv`):** Stores the persistent data for Students, Courses, etc.
- **Test Files (`test_*.c`):** Contains unit and integration tests (not linked in the main release build).

9 Conclusion

9.1 Accomplishments

In this project, we successfully designed and implemented a fully functional Student Information System in C. The system meets all core requirements, including:

- A robust file-based database system using CSV serialization.
- Dynamic memory management capable of handling growing datasets without recompilation.
- A relational data model enforcing integrity between Students, Courses, and Enrollments.
- A user-friendly command-line interface with comprehensive input validation.

9.2 Challenges Overcome

The development process presented several significant challenges:

- **Memory Management:** Managing pointers for dynamic arrays, especially passing them to functions that might call `realloc`, was difficult. We overcame this by strictly using double pointers (`Student **`) for all modification functions.
- **Circular Dependencies:** The relationship between `Course` and `Enrollment` caused header inclusion loops. This was solved using forward declarations in the header files.
- **Parsing Logic:** Handling CSV edge cases (like empty fields or fields with spaces) required writing a custom robust tokenizer instead of relying on the destructive `strtok`.

9.3 Lessons Learned

This project reinforced the importance of:

- **Modular Design:** Breaking the system into small, manageable files made debugging significantly easier.
- **Testing:** Writing unit tests early saved hours of debugging time by catching logic errors before they propagated to the UI.
- **Defensive Programming:** Always validating inputs and checking for NULL pointers prevents the majority of runtime crashes.

9.4 Future Improvements

Given more time, the system could be enhanced by:

- Implementing a Binary Search Tree (BST) or Hash Table index for $O(\log n)$ or $O(1)$ search performance instead of $O(n)$.
- Adding a Graphical User Interface (GUI) using a library like GTK or Qt.
- Implementing role-based access control (Admin vs. Student login).

A Appendices

A.1 Appendix A: Sample Console Output

```
Welcome to the Student Information System
==========================================
1. Student Management
2. Course Management
3. Enrollment Management
4. Grading System
5. Reports
0. Exit
Selection: 1

--- Student Management ---
1. Add Student
2. List Students
3. Search Student
4. Delete Student
0. Back
Selection: 2

ID        Name                 Major
----------------------------------------
2001001   John Doe             Computer Eng.
2001002   Jane Smith           Electrical Eng.
```

A.2 Appendix B: Valgrind Full Report

```
==5882== Memcheck, a memory error detector
==5882== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==5882== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright
   info
==5882== Command: ./programim
==5882== Parent PID: 4002
==5882==
==5882==
==5882== HEAP SUMMARY:
==5882==     in use at exit: 0 bytes in 0 blocks
==5882==   total heap usage: 38 allocs, 38 frees, 12,925 bytes allocated
==5882==
==5882== All heap blocks were freed -- no leaks are possible
==5882==
==5882== For lists of detected and suppressed errors, rerun with: -s
==5882== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Listing 1: Valgrind Output Summary

B References

1. , *Problem Solving and Program Design in C*, 2nd ed. Elliot Koffman and Jeri R. Hanly, 2010.

2. "Valgrind User Manual," *Valgrind.org*. [Online]. Available: https://valgrind.org.html. [Accessed: Jan. 14, 2026].

3. Interactions with ChatGPT (OpenAI) and Gemini (Google) for debugging, boilerplate generation, and documentation assistance throughout the development period (Dec 2025 - Jan 2026).